

Taller 1: Componentes Core y Estilizado

Objetivo: Extender la aplicación de React Native para consolidar el uso de Flexbox, la reutilización de Componentes (usando Props), el manejo de Áreas Seguras (`SafeAreaView`) y la integración de interfaces generadas por IA. El taller consiste en:

1. Crear una pantalla dividida en dos: un componente superior (Header Corporativo) creado a mano que muestre el logo de la empresa y una descripción, y un componente inferior que muestre una lista estática (FlatList) del directorio de empleados.
2. Todo debe renderizarse en la pantalla principal (`App.tsx`) sin el uso de navegación.

ESTRUCTURA DE CARPETAS DEL PROYECTO

Los estudiantes incorporarán los nuevos componentes en la estructura actual del proyecto de la siguiente manera:

```
TypeScript
mi-primer-app/
├── src/
│   ├── components/
│   │   ├── CompanyHeader.tsx    <-- [NUEVO] Encabezado con
imagen y descripción (Estilos con IA)
│   │   ├── ProfileCard.tsx     <-- Tarjeta individual para
mostrar el dato en la lista
│   │   └── EmployeeList.tsx    <-- Lista que renderiza
múltiples ProfileCard
│   └── App.tsx                 <-- Ensambla CompanyHeader y
EmployeeList controlando las áreas seguras y proporciones
```

PASO A PASO DEL TALLER

Paso 1: Crear el Encabezado Corporativo (`src/components/CompanyHeader.tsx`) Construye a mano un nuevo componente funcional que contenga una `Image` (el logo de tu empresa ficticia) y un `Text` con una breve descripción o eslogan debajo.

- **El Reto de la IA (Solo Estilos):** Puedes generar los estilos con IA
- **Prompt Sugerido:** "Tengo un componente de React Native con un View principal, una Imagen (logo) y un Text (eslogan). Escribeme solo el `StyleSheet.create({...})` para que la imagen sea circular de 120x120, esté todo centrado horizontal y verticalmente, y el eslogan tenga fuente en negrita, color gris oscuro y un poco de margen superior. Que se vea muy moderno."
- Copia los estilos generados por la IA y aplícalos a tus etiquetas (`style={styles.logo}`, etc.).



Paso 2: Reutilizar tus Componentes de Lista Asegúrate de tener listos y funcionando los componentes que creamos en clase:

- El componente funcional `ProfileCard` que recibe propiedades (`nombre` y `cargo`).
- El componente contenedor `EmployeeList` que utiliza `FlatList` y renderiza múltiples `ProfileCard` usando datos estáticos.

Paso 3: Ensamblar todo con Flexbox y Áreas Seguras (`App.tsx`) Limpia tu `App.tsx` y ensambla los dos componentes principales (`CompanyHeader` y `EmployeeList`).

- **Requisito de Áreas Seguras:** El contenedor principal debe ser un `SafeAreaProvider` y envolver un `SafeAreaView` para proteger la UI de la cámara o notch. Controla el color del `StatusBar`.
- **El Reto de Flexbox:** Usando la propiedad `flex`, distribuye el espacio de la pantalla para que el encabezado (`CompanyHeader`) ocupe exactamente el **30% superior** de la pantalla, y la lista (`EmployeeList`) ocupe el **70% restante** hacia abajo.



ENTREGABLES REQUERIDOS (2 CAPTURAS DE PANTALLA)

Deberás enviar directamente al **grupo de WhatsApp** del curso lo siguiente:

- **Captura 1 (Diseño en Expo Go):** Captura de pantalla desde tu celular o emulador demostrando la aplicación en funcionamiento. Se debe apreciar claramente:
 - La barra de estado (`StatusBar`) coloreada y el área segura respetada.
 - La correcta división en pantalla de 30% (Dashboard visual) y 70% (Lista de Empleados).
- **Captura 2 (Código Estructural):** Captura de tu editor de código (VS Code) mostrando específicamente el archivo `App.tsx` donde se evidencie el uso de Flexbox (los porcentajes) para dividir los componentes y el uso del `SafeAreaProvider`.

- Capturas enviadas al grupo de WhatsApp
- Repositorio de GitHub

